

Kapitel 18. Stränghantering

Innehållsförteckning

[Vad är `string` egentligen](#)[Skapa strängar](#)[Accessera enskilda tecken](#)[Tilldelning och jämförelse](#)[Insättning av text](#)[Söka text i strängar](#)[Ersätta text](#)[Substrängar](#)[Streams och strängar](#)[Manipulera tecken](#)[Konvertera till C-strängar](#)[Iteratorer och `string`](#)

Detta kapitel går djupare in på hur strängar kan hanteras i C++. Datatypen `string` är mycket flexibel och tillåter många fler operationer än de vi hittills använt.

Vad är `string` egentligen

Vi har hittills använt `string` som en primitiv datatyp, men det är egentligen frågan om en ganska avancerad *klass*. Redan av att man alltid måste inkludera headerfilen `<string>` för att använda strängar berättar att det är någonting speciellt med `string`.

Vi kan tänka oss att `string` är en vektor av `char`, d.v.s. vanliga tecken. Förutom att den är en abstraktion av en teckenvektor innehåller klassen även diverse metoder och överlagrade operatorer (se [Kapitel 21](#)) som gör det enklare att manipulera och använda dem. För dem som använt normala C-strängar av typen `char *` är klassen `string` säkert en välkommen nyhet. Många av de vanligaste felen i gamla C-program härstammar från felaktig användning av C-strängar. Ofta är dessa fel fatala och svåra att lokalisera. Det rekommenderas att man använder `string` i nya program om man inte har tungt vägande skäl till att hålla fast vid C-strängar.

För att kunna använda `string` måste man alltså inkludera headerfilen `<string>`. Utan denna är klassen inte definierad.